Comparative Study of Work-Stealing Schedulers

ABP, Chase-Lev, and Global Queue Scheduling

Aditya Varun V (AI22BTECH11001)
Anirudh Saikrishnan (CS22BTECH11004)
Monish Asawa (MA22BTECH11014)

Indian Institute of Technology Hyderabad

April 2026

Outline

1. Work Stealing
2. Schedulers to Implement
3. Benchmark Problem
4. Expected Results
5. Conclusion

Motivation: Scheduling Parallel Tasks

Many parallel programs generate tasks dynamically.

Examples:

- Recursive divide-and-conquer algorithms
- Parallel sorting
- Tree search and backtracking
- Branch-and-bound algorithms

Main challenge:

- The amount of work in each branch is often unpredictable.
- Some workers may finish early while others remain overloaded.
- This causes **load imbalance**.

What is Work Stealing?

Work stealing is a **decentralized scheduling strategy** for parallel computation.

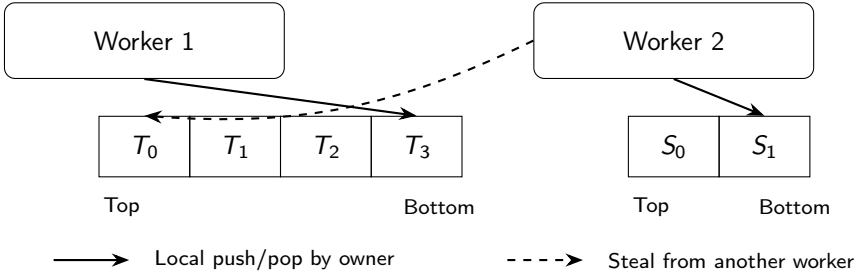
Core idea:

- Each worker maintains its own local deque of tasks.
- Workers execute tasks from their own deque.
- If a worker becomes idle, it steals tasks from another worker.

Key property:

- Load balancing happens dynamically.
- There is no central scheduler deciding all assignments.

Work Stealing: Deque Structure



Important convention:

- Owner works at the **bottom**.
- Thieves steal from the **top**.

Why Use Opposite Ends?

The owner and thieves access different ends of the deque.

Owner uses bottom

- Pushes newly generated tasks
- Pops most recent local task
- Gives LIFO behavior
- Improves cache locality

Thief uses top

- Steals older tasks
- Older tasks are often larger
- Reduces interference with owner
- Helps balance work efficiently

Result: most operations are local and uncontended.

Work Stealing vs Work Sharing

Work Sharing

- Busy workers push tasks to others.
- Load balancing is proactive.
- May move work unnecessarily.
- Can introduce extra synchronization.

Work Stealing

- Idle workers pull tasks from others.
- Load balancing is reactive.
- Work moves only when needed.
- Usually lower overhead.

Work stealing is especially effective for irregular parallel workloads.

Theoretical Intuition

Let:

$$T_1 = \text{total work}, \qquad T_\infty = \text{critical path length}, \qquad P = \text{number of processors}$$

For well-structured multithreaded computations, work stealing has expected running time:

$$T_P \approx \frac{T_1}{P} + O(T_\infty)$$

Interpretation:

- $\frac{T_1}{P}$ is the ideal parallel execution time.
- $O(T_\infty)$ is the unavoidable dependency cost.
- Stealing overhead is small when tasks are not too fine-grained.

Three Implementations

We implement and compare three scheduling strategies.

1. **Single Global Queue**
   - Baseline centralized scheduler
2. **ABP Work-Stealing Deque**
   - Bounded array-based work-stealing scheduler
3. **Chase-Lev Work-Stealing Deque**
   - Dynamic circular work-stealing deque

Goal: compare scalability, overhead, robustness, and load balance.

Baseline: Single Global Queue

Design:

- All generated tasks are inserted into one shared queue.
- All workers take tasks from this same queue.
- Synchronization is required for every push and pop.

Advantages:

- Very simple implementation.
- Naturally balances work across workers.

Limitations:

- High lock contention.
- Poor cache locality.
- Central queue becomes a scalability bottleneck.

Pseudocode: Global Queue Baseline

```
shared Queue Q
shared lock Q_lock
shared atomic active_tasks

worker_loop():
    while true:
        lock(Q_lock)
        if Q is not empty:
            task = Q.pop()
        else:
            task = EMPTY
        unlock(Q_lock)
        if task == EMPTY:
            if active_tasks == 0:
                terminate
            continue

        execute(task)

spawn(task):
    active_tasks++
    lock(Q_lock)
    Q.push(task)
    unlock(Q_lock)
```

ABP Work-Stealing Scheduler

Proposed by: Arora, Blumofe, and Plaxton.

Main structure:

- Each worker owns a fixed-size deque.
- Owner pushes and pops at the bottom.
- Thieves steal from the top.
- Synchronization uses atomic compare-and-swap.

Strengths:

- Low contention during local execution.
- Efficient for moderate and predictable task counts.

Weakness:

- Fixed capacity can cause overflow under high task generation.

Pseudocode: ABP Push and Steal

```
structure ABPDeque:
    array A[CAPACITY]
    atomic top
    integer bottom

pushBottom(x):
    b = bottom
    t = top.read()

    if b - t >= CAPACITY:
        return OVERFLOW

    A[b mod CAPACITY] = x
    memory_fence()
    bottom = b + 1
    return OK
```

```
steal():
    t = top.read()
    memory_fence()
    b = bottom

    if b <= t:
        return EMPTY

    x = A[t mod CAPACITY]

    if CAS(top, t, t + 1):
        return x
    else:
        return ABORT
```

Pseudocode: ABP Pop

```
popBottom():
    b = bottom - 1
    bottom = b
    memory_fence()

    t = top.read()
    size = b - t

    if size < 0:
        bottom = t
        return EMPTY

    x = A[b mod CAPACITY]
```

```
    if size > 0:
        return x

    // last-item race
    if CAS(top, t, t + 1):
        bottom = t + 1
        return x
    else:
        bottom = top.read()
        return EMPTY
```

Chase-Lev Work-Stealing Deque

Chase-Lev improves the ABP design by using a **dynamic circular array**.

Main structure:

- Each worker owns a deque.
- The deque is backed by a circular array.
- The array grows when it becomes full.
- Top and bottom remain monotonic counters.

Advantages over ABP:

- No fixed capacity limitation.
- Better robustness for unpredictable workloads.
- Better memory use than choosing a large fixed array.

Pseudocode: Chase-Lev Push

```
structure ChaseLevDeque:
    atomic top
    integer bottom
    CircularArray array

pushBottom(x):
    b = bottom
    t = top.read()
    a = array

    if b - t >= a.size - 1:
        a = grow(a, t, b)
        array = a

    a.put(b, x)
    memory_fence()
    bottom = b + 1
    return OK
```

```
grow(a, top, bottom):
    new_array =
        CircularArray(2 * a.size)

    for i from top to bottom - 1:
        new_array.put(i, a.get(i))

    return new_array
```

Pseudocode: Chase-Lev Steal and Pop

```
steal():
    t = top.read()
    memory_fence()
    b = bottom
    a = array

    if b <= t:
        return EMPTY

    x = a.get(t)

    if CAS(top, t, t + 1):
        return x
    else:
        return ABORT

popBottom():
    b = bottom - 1
    bottom = b
    memory_fence()
```

```
    t = top.read()
    a = array

    if b < t:
        bottom = t
        return EMPTY

    x = a.get(b)

    if b > t:
        return x

    if CAS(top, t, t + 1):
        bottom = t + 1
        return x
    else:
        bottom = t + 1
        return EMPTY
```

Algorithm Comparison

Property	Global Queue	ABP	Chase-Lev
Queue design	Single shared queue	Per-worker deque	Per-worker deque
Capacity	Dynamic	Fixed	Dynamic
Contention	High	Low	Low
Locality	Weak	Strong	Strong
Overflow risk	No	Yes	No practical fixed limit
Implementation	Simple	Moderate	More complex
Expected scaling	Weak	Good	Best under load

Chosen Problem: N-Queens Backtracking

We choose **N-Queens** as the main benchmark problem.

Problem statement:

- Place N queens on an $N \times N$ chessboard.
- No two queens may attack each other.
- Count all valid board configurations.

Why this is a good benchmark:

- Naturally recursive.
- Highly irregular search tree.
- Branches terminate at different depths due to pruning.
- Task granularity can be controlled using split depth.

Why N-Queens Tests Work Stealing Well

N-Queens creates an uneven tree of computation.

- Some partial boards lead to many valid continuations.
- Some partial boards are pruned almost immediately.
- A static assignment of branches can create severe imbalance.

Expected effect:

- Workers with short branches become idle.
- They steal larger remaining branches from busy workers.
- Utilization improves compared with static or centralized scheduling.

Pseudocode: N-Queens Task Generation

```
solve(row, board):
    if row == N:
        atomic solution_count++
        return

    for col in 0 to N - 1:
        if safe(board, row, col):
            new_board =
                board with queen
                at (row, col)

            if row < SPLIT_DEPTH:
                spawn(Task(row + 1,
                           new_board))
            else:
                solve(row + 1, new_board)
```

Split depth controls task granularity:

- Small split depth: fewer, larger tasks.
- Large split depth: many, smaller tasks.

Purpose:

- Exposes parallel branches.
- Allows scheduler comparison.

Experimental Metrics

We will compare all three schedulers using the same N-Queens workload.

Performance metrics:

- Total execution time
- Speedup over sequential execution
- Throughput, measured as tasks completed per second

Scheduler metrics:

- Number of successful steals
- Number of failed steal attempts
- Worker idle time
- Load imbalance across workers
- Queue overflow count for ABP
- Number of resize operations for Chase-Lev

Expected Result 1: Global Queue

Expected behavior:

- Performs reasonably for small thread counts.
- Becomes slower as more workers compete for the same queue.
- Lock contention increases with thread count.
- Cache locality is weaker because tasks migrate through one shared structure.

Prediction:

- Global queue will be the easiest to implement.
- It will likely scale the worst.

Expected Result 2: ABP Scheduler

Expected behavior:

- Low overhead for local push and pop operations.
- Good scalability when queue capacity is sufficient.
- Stealing overhead remains limited because stealing happens only when workers are idle.

Potential weakness:

- Fixed-size deques may overflow for high task volume.
- Capacity tuning becomes important.

Prediction:

- ABP should outperform the global queue under moderate workloads.
- It may degrade or fail when the task frontier becomes too large.

Expected Result 3: Chase-Lev Scheduler

Expected behavior:

- Similar low-contention benefits as ABP.
- Handles large and varying task frontiers more robustly.
- Dynamic resizing avoids fixed-capacity failure.

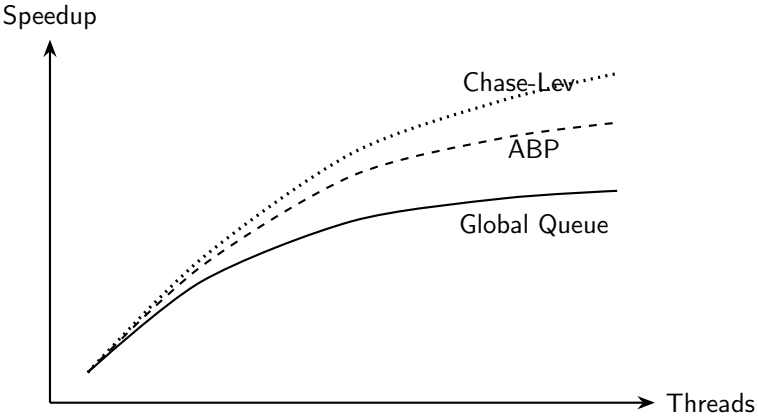
Trade-off:

- More implementation complexity.
- Occasional resizing cost during execution.

Prediction:

- Chase-Lev should perform best for irregular high-load cases.
- It should be more robust than ABP when task generation is unpredictable.

Expected Performance Trends



Expected trend:

- Global queue saturates earliest.
- ABP scales well until capacity pressure appears.

Effect of Task Granularity

Task granularity is controlled by **split depth** in N-Queens.

Too coarse

- Few tasks generated.
- Not enough parallelism.
- Some workers remain idle.

Too fine

- Too many small tasks.
- Scheduling overhead dominates.
- Steal attempts increase.

Best performance is expected at intermediate granularity.

Summary

- Work stealing is a decentralized method for balancing irregular parallel workloads.
- Each worker owns a deque, while idle workers steal from others.
- We compare three designs:
  - Single global queue
  - ABP bounded work-stealing deque
  - Chase-Lev dynamic circular work-stealing deque
- N-Queens is chosen because it creates an irregular recursive search tree.
- Chase-Lev is expected to be the most robust under high and varying task volume.

References

- R. D. Blumofe and C. E. Leiserson, "Scheduling multithreaded computations by work stealing," *Journal of the ACM*, vol. 46, no. 5, pp. 720–748, 1999.
- N. S. Arora, R. D. Blumofe, and C. G. Plaxton, "Thread scheduling for multiprogrammed multiprocessors," *SPAA*, 2001.
- D. Chase and Y. Lev, "Dynamic circular work-stealing deque," *SPAA*, 2005.